

参考答案

1、段地址默认为 ds

在编译器中可以用 dx:[0000]表示，在 debug 中 dx:[0000]报错，可以用 dx: 、 [0000]两条命令代替

2、（1）寄存器寻址

（2）寄存器间接寻址 10300H

（3）寄存器间接寻址 20200H

（4）直接寻址 10060H

（5）基址变址寻址 10500H

（6）基址变址相对寻址 10306H

（7）直接寻址 11000H

（8）寄存器相对寻址 10306H

（9）直接寻址 10065H

3、对 错 错 对 错 错 错 错 错

4、

	al	cf	of	sf	zf
xor al,al	00	0	0	0	1
mov al,255	ff	0	0	0	1
inc al	00	0	0	0	1
not al	ff	0	0	0	1
shl al,1	fe	1	0	1	0
cmp sl,10	fe	0	0	1	0
sub al,7fh	7f	0	1	0	0
shl al,1	fe	0	1	1	0

5、（1）传送立即数 1000H；传送偏移地址 1000H 中的数据

（2）传送 mem 中的数据；传送 mem 代表的偏移地址（必须是静态）

（3）传送 mem 中的数据；传送 mem 代表的偏移地址（可以是动态如 lea ax,mem[si]）

（4）短转移 (IP)=(IP)+8 位位移；段内近转移 (IP)=(IP)+16 位位移

（5）仅改变标志位；改变标志位，结果送入 dx 寄存器中

（6）cl 送入 ds:[bp+si]中，指令长度 2 字节；功能相同，指令长度 3 字节

6、栈顶为 2FF00H，push 后栈顶为 2FEFEH 内容为 56H 和 34H

7、

	ax	cf	zf	of	sf	pf
xor ax,value	396BH	0	0	0	0	0
and ax,value	0010H	0	0	0	0	0
sub ax,value	0F965H	1	0	0	1	1
cmp ax,value	1978H	1	0	0	1	1
not ax	0E687H	0	0	0	0	0
test ax,value	1987H	0	0	0	0	0

8、是否转为机器指令并执行

9、

```
0770:0000 22 34 47 4F 4F 44 01 00-00 01 00 00 78 56 44 43 "4G00D.....xJDC
0770:0010 12 00 64 00 64 00 ..d.d.
```

10、 data segment

```
db1 db 2 dup(1,2,5 dup(3),4)
```

```
db2 db 'STUDENTS$'
```

```
db3 db 12h,0abh,0cdh
```

```
count equ 16
```

```
data ends
```

11、 assume cs:code,ds:data

```
data segment
```

```
string db 'ad','$'
```

```
data ends
```

```
code segment
```

```
start: mov ax,data
```

```
mov ds,ax
```

```
mov si,0ffffh
```

```
c1: inc si
```

```
cmp string[si],'$'
```

```
jne c1
```

```
mov word ptr string+1[si],si
```

```
mov ah,4ch
```

```
int 21h
```

```
code ends
```

```
end start
```

12、 assume cs:code,ds:data,ss:stack

```
data segment
```

```
db 96h
```

```
data ends
```

```
stack segment
```

```
db 16 dup(0)
```

```
stack ends
```

```
code segment
```

```
start: mov ax,data
```

```
mov ds,ax
```

```
mov ax,stack
```

```
mov ss,ax
```

```
mov sp,16
```

```
mov al,ds:[0]
```

```
push ax
```

```
mov cl,4
```

```
shr al,cl
```

```
add al,30h
```

```

        mov byte ptr ds:[1],al
        pop ax
        and al,0fh
        add al,30h
        mov byte ptr ds:[2],al
        mov ah,4ch
        int 21h
code ends
end start
13、 assume ds:data,ss:stack,cs:code
data segment
        db 12h,56h,9ah,0deh
        buf db 8 dup(?)
data ends
stack segment
        dw 8 dup(0)
stack ends
code segment
start:   mov ax,data
        mov ds,ax
        mov ax,stack
        mov ss,ax
        mov sp,16
        lea bx,buf
        mov di,0
        mov si,0
        mov cx,4
s1:      push cx
        mov al,[di]
        inc di
        mov dx,ax
        mov cl,4
        shr al,cl
        mov buf[si],al
        inc si
        and dl,0fh
        mov buf[si],dl
        inc si
        pop cx
        loop s1
        mov ah,4ch
        int 21h
code ends
end start

```

14、 assume cs:code

code segment

```
start:  mov cx,1
s:      mov ah,07
        int 21h
        mov dl,al
        cmp dl,'A'
        jb c1
        cmp dl,'Z'
        jna c2
        cmp dl,'a'
        jb c1
        cmp dl,'z'
        jna c3
c3:     sub dl,20h
c2:     mov ah,2
        int 21h
c1:     inc cx
        loop s
```

code ends

end start

15、 assume ds:data,ss:stack,cs:code

data segment

```
infor db 'please input:',0dh,0ah,'$'
error db 0dh,0ah,'wrong number','$'
buf db 4
    db ?
    db 4 dup(?)
s6 db 0
s7 db 0
s8 db 0
s9 db 0
s10 db 0
```

data ends

stack segment

```
db 32 dup(0)
```

stack ends

code segment

```
start:  mov ax,data
        mov ds,ax
        mov ax,stack
        mov ss,ax
        mov sp,32
        lea dx,infor
```

```

        mov ah,9h
        int 21h
        mov cx,10
c1:     lea dx,buf
        mov ah,0ah
        int 21h
        call judgement
        mov dl,0ah
        mov ah,02h
        int 21h
        loop c1
        call display
        mov ah,4ch
        int 21h
judgement proc near
        push cx
        mov cl,[buf+1]
        mov ch,0
        cmp cx,3
        je j2
        cmp cx,2
        je j5
        cmp cx,1
        je j3
j1:     lea dx,error
        mov ah,9h
        int 21h
        pop cx
        inc cx
        jmp j4
j2:     mov al,[buf+2]
        cmp al,'1'
        jne j1
        mov al,[buf+3]
        cmp al,'0'
        jne j1
        mov al,[buf+4]
        cmp al,'0'
        jne j1
        inc byte ptr s10
j3:     pop cx
j4:     ret
j5:     mov al,[buf+2]
        cmp al,'6'

```

```

        je j6
        cmp al,'7'
        je j7
        cmp al,'8'
        je j8
        cmp al,'9'
        je j9
        jmp j3
j6:      inc byte ptr s6
        jmp j3
j7:      inc byte ptr s7
        jmp j3
j8:      inc byte ptr s8
        jmp j3
j9:      inc byte ptr s9
        jmp j3
judgement endp
display proc near
        mov si,0
        mov cx,5
s:       mov dl,s6[si]
        add dl,30h
        mov ah,02h
        int 21h
        mov dl,09h
        mov ah,02h
        int 21h
        inc si
        loop s
        ret
display endp
code ends
end start

```